Akademia Nauk Stosowanych w Nowym Sączu
Wydział Nauk Inżynieryjnych • Informatyka • Studia I stopnia

Niezawodność systemów informatycznych

Zajęcia projektowe nr 3, blok P2

Testy strukturalne: testy mutacyjne

1 Informacje organizacyjne

- **Blok tematyczny z karty zajęć:** P2: Testy strukturalne
- **Punkt wyjścia:** testy jednostkowe napisane na zajęciach nr 2

Cel dzisiejszych zajęć

Po dzisiejszych zajęciach każdy zespół powinien:

1. rozumieć, czym jest mutacja kodu i jak powstaje *mutation score*,
2. mieć uruchomione narzędzie do testów mutacyjnych na własnym wariancie SUT,
3. umieć odróżnić mutanta zabitego przez testy od mutanta, który przetrwał z powodu brakującej asercji, od mutanta równoważnego (equivalent mutant),
4. poprawić swoje testy tak, aby podnieść mutation score tam, gdzie to zasadne.

Dlaczego to osobne zajęcia, a nie część poprzednich

Pokrycie kodu (zajęcia nr 2) mówi, co zostało wykonane. Testy mutacyjne mówią, czy to, co zostało wykonane, zostało też rzeczywiście sprawdzone. To różne pytania i wymagają osobnego narzędzia oraz osobnej interpretacji wyniku.

2 Wprowadzenie teoretyczne

Mutacja kodu, mutant

Mutacja to drobna, celowa zmiana w kodzie źródłowym, np. zamiana `>` na `>=`, zamiana `+` na `-`, albo zmiana stałej liczbowej. Kod po takiej zmianie nazywa się mutantem. Narzędzie do testów mutacyjnych generuje wielu mutantów automatycznie, każdego z jedną zmianą, i uruchamia na każdym cały zestaw testów.

Zabity mutant, przetrwały mutant

Jeśli po wprowadzeniu mutacji choć jeden test zawiedzie (wykryje różnicę w zachowaniu), mutant jest *zabity*. Jeśli wszystkie testy nadal przechodzą mimo zmiany w kodzie, mutant *przetrwał*, co oznacza, że testy nie są wystarczająco czułe, żeby wykryć tę konkretną zmianę.

Mutation score

$$\text{mutation score} = \frac{\text{liczba zabitych mutantów}}{\text{liczba wszystkich mutantów}}$$

Im wyższy wynik, tym silniejsze są testy w wykrywaniu drobnych błędów logicznych. W odróżnieniu od pokrycia kodu, mutation score mówi coś o jakości asercji, nie tylko o tym, który kod został wykonany.

Equivalent mutant

Czasem mutacja nie zmienia faktycznego zachowania programu dla żadnych możliwych danych wejściowych, np. zamiana warunku na logicznie równoważny. Taki mutant nigdy nie zostanie zabity, niezależnie od jakości testów, i nie powinien obniżać oceny testów. Rozpoznanie equivalent mutant wymaga ręcznej analizy, narzędzia nie potrafią tego zrobić automatycznie w każdym przypadku.

Testy mutacyjne jako test na testy

Testy mutacyjne odwracają pytanie, które zwykle się zadaje. Zamiast pytać, czy kod działa poprawnie, pytają, czy testy w ogóle zauważyłyby, gdyby kod działał niepoprawnie. To dlatego czasem nazywa się je metatestami, testami testujących testy.

2.1 Narzędzie

pytest-gremlins

`pytest-gremlins` to wtyczka do pytest realizująca testy mutacyjne. Generuje mutantów, uruchamia na każdym istniejący zestaw testów i raportuje, które mutanty przetrwały. Działa natywnie na Windows, Linuksie i macOS, bez potrzeby żadnego dodatkowego środowiska, a w podstawowym użyciu nie wymaga pliku konfiguracyjnego: uruchamia się jako zwykła opcja polecenia `pytest`, które zespół i tak już zna z zajęć nr 2.

Podstawowe użycie pytest-gremlins

```
pip install pytest-gremlins

pytest --gremlins --gremlin-targets=zespol_A.py test_zespol_A.py
```

Polecenie najpierw sprawdza, czy dotychczasowe testy przechodzą (testy mutacyjne mają sens tylko na zielonym punkcie wyjścia), a następnie generuje mutantów modułu `zespol_A.py` i uruchamia na każdym test `test_zespol_A.py`. Wynik w terminalu wypisuje, ile mutantów zostało zabitych (w dokumentacji narzędzia opisywane jako *zapped*) i ile przetrwało. Rozszerzony raport można uzyskać flagą `--gremlin-report=html`.

3 Zadanie projektowe na dziś

1. **Uruchomcie narzędzie do testów mutacyjnych** na swoim wariancie SUT, korzystając z testów napisanych na zajęciach nr 2.
2. **Policzcie mutation score** dla przetestowanych metod.
3. **Przeanalizujcie co najmniej pięciu przetrwałych mutantów**: dla każdego ustalcie, czy to equivalent mutant, czy brakujący przypadek testowy.

4. **Dopiszcie testy**, które zabijają te spośród przetrwałych mutantów, które nie są equivalent, i policzcie mutation score ponownie.
5. **Zanotujcie**, ile mutantów uznaliście za equivalent i dlaczego.

Częsty błąd

Dopisywanie testu, który zabija konkretnego mutanta, ale nie sprawdza niczego sensownego merytorycznie, np. test dobrany tak, żeby akurat trafić na zmienioną stałą, bez związku z rzeczywistym scenariuszem użycia metody. Taki test podnosi wynik, ale nie poprawia jakości testów w praktyce.

Co trafia do raportu

Na koniec dzisiejszych zajęć zespół dopisuje do wspólnego raportu projektowego:

- mutation score przed i po dzisiejszej pracy,
- listę przeanalizowanych przetrwałych mutantów z klasyfikacją (equivalent / brakujący test) i uzasadnieniem,
- opis dopisanych testów i to, jakich mutantów zabijają,
- wnioski: czy praca nad testami mutacyjnymi ujawniła coś, czego nie widać było przy samym pokryciu kodu.

4 Pytania kontrolne

1. Czym różni się mutation score od pokrycia kodu? Dlaczego oba mają sens mierzone razem?
2. Co to jest equivalent mutant i dlaczego utrudnia interpretację mutation score?
3. Podaj przykład operatora mutacji i opisz, jaki test wykryłby wprowadzoną przez niego zmianę.
4. Dlaczego wysoki mutation score jest trudniejszy do osiągnięcia niż wysokie pokrycie gałęzi?

5 Przygotowanie do zajęć nr 4 (blok P3)

Zajęcia nr 4 i 5 dotyczą testów funkcjonalnych i niefunkcjonalnych, opartych na specyfikacji działania systemu, a nie na jego kodzie źródłowym. Na następne zajęcia:

- mieć dostęp do dokumentacji SUT (`sut_dokumentacja.pdf`), będzie podstawą do wyznaczania klas równoważności,
- zachować dotychczasowe testy w repozytorium, blok P3 je rozbudowuje, a nie zastępuje.